

PS2 键盘控制电子琴设计

1 、实验任务:

学习对 PS2 键盘数据程序的设计；掌握 PS2 键盘应用技术。

2 、实验原理:

PS/2 协议是一种双向同步串行通信协议，主机与外设通过时钟线同步，并通过数据线交换数据，且主机和外设都可以控制他们，但是在协议中，主机具有较高的优先级。此外，时钟总是由外设产生的，一般主机和外设传输数据的最大时钟频率为 33KHz，大多数 PS/2 外设工作在 10~20kHz。在协议中数据的传送格式为短帧格式（11 位或 12 位）。

下图 1.1 是 PS2 协议的时序图。读取基本的键盘数据，不需要 FPGA 发送任何数据，只需读取键盘发回来的数据即可。PS2 协议对数据的读取是“Clock 的下降沿”有效。每次键盘发送 11 个 clock 信号，我们需要做的事情就是在时钟的下降沿读取数据。

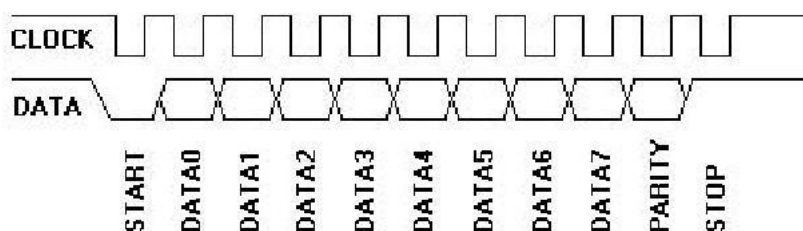


图 1.1 PS2 键盘的时序图

PS2 的一帧是 11 位。对 PS2 进行解码时，除了第 1~8 位数据位以外，其余的位都可以不做考虑。

键盘的编码有“通码”（Make）和“断码”（Break）之分。看得简单一点就是，“通码”是某按键的“按下事件”，“断码”是某按键的“释放事件”。通码和断码分别对应独立的编码。

在图 2.2 所示的组合模块 ps2_module.v 中包含了两个功能块：“电平检测模块”和“ps2 解码模块”。电平检测模块的添加是为了检查 PS2 时钟信号的“下降沿”，此外也是为了使模块的建立更简单。PS2 解码模块如字面上的意思，对每帧（11 位一组数据）的数据进行解码和过滤的行为，最后将一字节数据输出 检测到的数据然后“确认完成”产生一个高脉冲，表示完成一次性的操作。

我们已知 PS2 采用的是串行通信，且 PS2 一帧数据的第 0 位是开始位，所以无视。

在接下来的 8 个位都是数据位，PS2 的数据位是从最低位开始，最高位结束。由此可以对 PS2_Data_Pin_In 引脚进行读值转为并行数据并存储在 data_a 中。

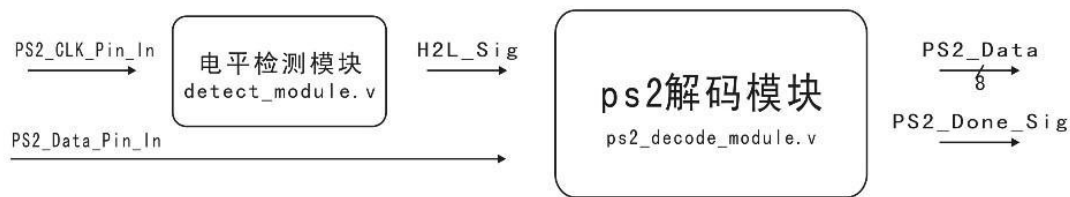


图 2.2 键盘解码组合模块图

图 2.3 是 PS2 键盘控制模型电子琴电路顶层设计。除了 PS2 通信模块 PS2_PIANO 和图 2.3 中的 CODE3 模块（例 11-2）等稍有不同外，此电路所有其他模块与电路功能与以上的实验十图 2.4 完全相同，工作原理也类似。对此不再重复说明。

例 11-1 是根据来自 PS2_PIANO 模块的键盘码，即图 2.4 的 PS2 键盘码设计的。PS/2 键盘接口是个 6 脚连接器。PS/2 其 4 个脚的功能分别是时钟端口、数据端口、+5V 电源端口和电源接地端口。PS/2 键盘依靠 PC 的 PS/2 端口提供+5V 电源。PS/2 是双端口双向通信模式，即遵循双向同步通信协议。通信的双方过时钟口同步，然后通过数据口进行数据通信。通信中，主机若要控制另一方通信选择，把时钟拉至低电平即可。

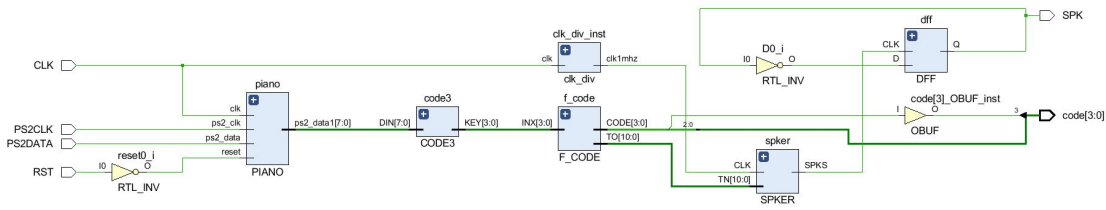


图 2.3 PS2 键盘控制模型电子琴电路 RTL 原理图

- PS2 通信过程中，数据以帧为单位进行传输，每帧包含 11~12 位数据，具体方式是：
- （1）数据的第 1 个位为起始位，其逻辑恒为 0。
 - （2）接下来是 8 个数据位，低位在前；时钟下降沿读取。
 - （3）然后是 1 个 1 位奇偶校验位，作奇校验；接下的第 11 位是停止位，恒为 1。
 - （4）必要时 1 个答应位，用于主机对设备的通信。
- 通信过程中当 PS/2 设备等待发送数据时，首先检查时钟端以便确认其高电平；如果为低电平，则认为是主机抑制了通信，此时必须缓存待发送的数据，直到获得总线控制权。如果时钟信号为高电平，则 PS/2 设备将数据发送给主机。由于 PS/2 通信协议是一种双向同步串行通信协议，数据可以从主机发往设备，也可以由设备发往主机。实际应用中，PS/2 键盘作为一种输入设备，都是由键盘往主机发送数据，主机读取数据。这时由 PS/2 键盘产生

时钟信号，发送数据时按照数据帧格式顺序发送。其中数据位在时钟为高电平时准备好，在时钟的下降沿主机就可以读取数据。因此通常情况只有键盘向主机发送数据，PS2 键盘的两个工作端口都是单向输出口。对于 PS2 通信的详细情况可参考相关资料。

Key	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O							
Data	1C	32	21	23	24	2B	34	33	43	3B	42	4B	3A	31	44							
Key	P	Q	R	S	T	U	V	W	X	Y	Z	0	1	2	3							
Data	4D	15	2D	1B	2C	3C	2A	1D	22	35	1A	45	16	1E	26							
Key	4	5	6	7	8	9	`	-	=	\	]	;	'	,	.							
Data	25	2E	36	3D	3E	46	0E	4E	55	5D	5B	4C	52	41	49							
Key	/	[	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10	F11	F12	KP0							
Data	4A	54	05	06	04	0C	03	0B	83	0A	01	09	78	07	70							
Key	KP1	KP2	KP3	KP4	KP5	KP6	KP7	KP8	KP9	KP-	KP+	KP/	KP*	END								
Data	69	72	7A	6B	73	74	6C	75	7D	71	7B	79	4A	7C	69							
Key	BKSP		SPACE		TAB		CAPS		L SHFT		L CTRL		L CUI		L ALT		R SHFT		R CTRL		R CUI	
Data	66		29		0D		58		12		14		1F		11		59		14		27	
Key	R ALT		APPS		ENTER		ESC		INSERT		HOME		PG UP		DELETE		PG DN		NUM			
Data	11		2F		5A		76		70		6C		7D		71		7A		77			
Key	U ARROW		L ARROW		D ARROW		R ARROW		KP EN		SCROLL		PRNT		SCRN		PAUSE					
Data	75		6B		72		74		5A		7E		12		7C		14					

图 2.4 PS2 键盘键控与输出码对照表

【例 1-1】顶层

```
// Create Date: 2021/03/18 16:25:53
// Design Name: PS2 键盘电子琴
// Module Name: ps2_piano
module ps2_piano(
CLK,
PS2DATA,
PS2CLK,
RST,
SPK,
code
);
input CLK;
input PS2CLK;
input PS2DATA;
input RST;
output SPK;
output [3:0] code;
wire [3:0] code;
wire c0;
clk_div clk_div_inst(.clk(CLK),.clk1mhz(c0));
```

```

wire [7:0] ps2DOUT;
PIANO piano(
.reset(!RST),
.clk(CLK),
.ps2_clk(PS2CLK),
.ps2_data(PS2DATA),
.ps2_data1(ps2DOUT)
);
wire [3:0] key;
CODE3 code3 (
.DIN(ps2DOUT),
.KEY(key)
);
wire H;
wire [10:0] TO;
F_CODE f_code (
.INX(key),
.CODE(code),
.H(H),
.TO(TO)
);
wire SPKS;
SPKER spker(
.CLK(c0),
.TN(TO),
.SPKS(SPKS)
);
wire Q;
DFF dff (
.CLK(SPKS),
.D(!Q),
.Q(Q)
);
assign SPK = Q;
endmodule

```

【例 1-2】键盘码控制

```

module PIANO (reset, clk, ps2_clk, ps2_data ,ps2_data1);
input reset; input clk; input ps2_clk; input ps2_data;
output[7:0] ps2_data1;
reg ps2_clk_s; reg enk; reg ps2_clk_r; reg flag; reg always_one;
reg[6:0] lowtimecount; reg[3:0] count;
reg[11:0] senddata;
integer TONE;

```

```

wire PreCLK; wire FullSpkS; wire ps2_data_s;

reg[7:0] ps2_data1; reg[7:0] ps2_data2; reg[7:0] ps2_data4;
parameter[10:0] ps2_f4 = 13'b10_11110100_0;
reg[8:0] q_data1_x; reg[8:0] q_data2_x; reg[8:0] q_data3_x; reg[8:0] q_data4_x;
reg[7:0] q_data1; reg[7:0] q_data2; reg[7:0] q_data3; reg[7:0] q_data4;
//-----
parameter
m2_st0=0,m2_new=1,m2_st1=2,m2_st2=3,m2_st3=4,m2_st4=5,m2_st5=6 ,m2_st6=
7 ;
parameter m2_st7=8,
m2_st8=9,m2_st9=10,m2_st10=11,m2_st11=12,m2_st12=13,m2_st13=14;
parameter
m2_st14=15,m2_st15=16,m2_st16=17,m2_st17=18,m2_st18=19,m2_st19=20,m2_st
20=21;
parameter
m2_st21=22,m2_st22=23,m2_st23=24,m2_st24=25,m2_st25=26,m2_st26=27,m2_st
27=28;
parameter
m2_st28=29,m2_st29=30,m2_st30=31,m2_st31=32,m2_st32=33,m2_st33=34,m2_st
34=35;
parameter
m2_st35=36,m2_st36=37,m2_st37=38,m2_st38=39,m2_st39=40,m2_st40=41,m2_st
41=42;
parameter m2_st42=43,m2_st43=44, m2_st44=45,m2_st45=46,m2_st46=47;
reg[5:0] m2_state; reg[5:0] m2_nexts; integer Tone2; reg[7:0] INDEXV;
//-----
assign ps2_data_s = senddata[0] ;
always @(reset) begin
if (reset==1'b0) flag <= 1'b1 ; end
always @(negedge clk or posedge reset) begin : u1
if (reset==1'b1) begin lowtimecount <= {7{1'b0}} ; enk <= 1'b0 ; ps2_clk_s <= 1'b1 ;
end else begin if (flag == 1'b1)
begin if (lowtimecount < 100)
begin lowtimecount<=lowtimecount+1 ; ps2_clk_s<=1'b0 ; end
else if (lowtimecount < 120)
begin lowtimecount <= lowtimecount + 1; enk <= 1'b1 ; end
else begin ps2_clk_s <= 1'b1 ; enk <= 1'b0 ; end
end end end
always @(negedge clk) begin : u2
ps2_clk_r <= ps2_clk ; end
always @(negedge ps2_clk_r or posedge enk) begin : u3
if (enk == 1'b1) begin senddata <= {1'b1, ps2_f4} ; end
else begin senddata[11:0] <= {1'b1, senddata[11:1]} ; end

```

```

end
//-----Receve part-----

always @(posedge enk or negedge ps2_clk_r) begin : con
if (enk == 1'b1) begin count <= {4{1'b0}} ; end
else begin if (count < 15) //receive at rising_edfe
begin count <= count + 1 ; end
end end
always @(posedge clk) begin : u4
m2_state <= m2_nexts ; end
always @(posedge ps2_clk_r) begin : u5
if (count < 10) begin m2_nexts <= m2_st0 ; end
else begin
case (m2_state)
m2_st0 : begin m2_nexts<=m2_st1; end
m2_st1 : begin m2_nexts<=m2_st3; end
m2_st3 : begin m2_nexts<=m2_st4; end//start
m2_st4 : begin m2_nexts<=m2_st5; q_data1_x<={ps2_data, q_data1_x[8:1]}; end
m2_st5 : begin m2_nexts<=m2_st6; q_data1_x<={ps2_data, q_data1_x[8:1]}; end
m2_st6 : begin m2_nexts<=m2_st7; q_data1_x<={ps2_data, q_data1_x[8:1]}; end
m2_st7 : begin m2_nexts<=m2_st8; q_data1_x<={ps2_data, q_data1_x[8:1]}; end
m2_st8 : begin m2_nexts<=m2_st9; q_data1_x<={ps2_data, q_data1_x[8:1]}; end
m2_st9 : begin m2_nexts<=m2_st10; q_data1_x<={ps2_data, q_data1_x[8:1]}; end
m2_st10 : begin m2_nexts<=m2_st11; q_data1_x<={ps2_data, q_data1_x[8:1]}; end
m2_st11 : begin m2_nexts<=m2_st12; q_data1_x<={ps2_data, q_data1_x[8:1]}; end
m2_st12 : begin m2_nexts<=m2_st13; q_data1_x<={ps2_data, q_data1_x[8:1]};
q_data1 <= q_data1_x[8:1]; end
m2_st13 : begin m2_nexts<=m2_st14; end
m2_st14 : begin m2_nexts<=m2_st15; end
m2_st15 : begin m2_nexts<=m2_st16; q_data2_x<={ps2_data, q_data2_x[8:1]}; end
//data0
m2_st16 : begin m2_nexts<=m2_st17; q_data2_x<={ps2_data, q_data2_x[8:1]}; end
m2_st17 : begin m2_nexts<=m2_st18; q_data2_x<={ps2_data, q_data2_x[8:1]}; end
m2_st18 :
begin
m2_nexts <= m2_st19 ;
q_data2_x <= {ps2_data, q_data2_x[8:1]};
always_one <= ps2_data ;
end
m2_st19 :
begin
m2_nexts <= m2_st20 ;
q_data2_x <= {ps2_data, q_data2_x[8:1]};
end
end

```

```

m2_st20 :
begin
m2_nexts <= m2_st21 ;
q_data2_x <= {ps2_data, q_data2_x[8:1]} ;
end
m2_st21 :
begin
m2_nexts <= m2_st22 ;
q_data2_x <= {ps2_data, q_data2_x[8:1]} ;
end
m2_st22 :
begin
m2_nexts <= m2_st23 ;
q_data2_x <= {ps2_data, q_data2_x[8:1]} ; //data7
end
m2_st23 :
begin
m2_nexts <= m2_st24 ;
q_data2_x <= {ps2_data, q_data1_x[8:1]} ;
q_data2 <= q_data2_x[8:1] ;
end
m2_st24 :
begin
if (always_one == 1'b1)
begin
m2_nexts <= m2_st25 ; //stop2
end
else
begin
m2_nexts <= m2_st14 ;
end
end
m2_st25 :
begin
m2_nexts <= m2_st26 ;
end
m2_st26 :
begin
m2_nexts <= m2_st27 ;
q_data3_x <= {ps2_data, q_data3_x[8:1]} ; //data0
end
m2_st27 :
begin
m2_nexts <= m2_st28 ;

```

```

q_data3_x <= {ps2_data, q_data3_x[8:1]} ;
end
m2_st28 :
begin
m2_nexts <= m2_st29 ;
q_data3_x <= {ps2_data, q_data3_x[8:1]} ;
end
m2_st29 :
begin
m2_nexts <= m2_st30 ;
q_data3_x <= {ps2_data, q_data3_x[8:1]} ;
end
m2_st30 :
begin
m2_nexts <= m2_st31 ;
q_data3_x <= {ps2_data, q_data3_x[8:1]} ;
end
m2_st31 :
begin
m2_nexts <= m2_st32 ;
q_data3_x <= {ps2_data, q_data3_x[8:1]} ;
end
m2_st32 :
begin
m2_nexts <= m2_st33 ;
q_data3_x <= {ps2_data, q_data3_x[8:1]} ;
end
m2_st33 :
begin
m2_nexts <= m2_st34 ;
q_data3_x <= {ps2_data, q_data3_x[8:1]} ; //data7
end
m2_st34 :
begin
m2_nexts <= m2_st35 ;
q_data3_x <= {ps2_data, q_data1_x[8:1]} ;
q_data3 <= q_data3_x[8:1] ;
end
m2_st35 :
begin
m2_nexts <= m2_st36 ; //stop3
end
m2_st36 :
begin

```

```

m2_nexts <= m2_st37 ; //start
end
m2_st37 :
begin
m2_nexts <= m2_st38 ;
q_data4_x <= {ps2_data, q_data4_x[8:1]} ; //data0
end
m2_st38 :
begin
m2_nexts <= m2_st39 ;
q_data4_x <= {ps2_data, q_data4_x[8:1]} ;
end
m2_st39 :
begin
m2_nexts <= m2_st40 ;
q_data4_x <= {ps2_data, q_data4_x[8:1]} ;
end
m2_st40 :
begin
m2_nexts <= m2_st41 ;
q_data4_x <= {ps2_data, q_data4_x[8:1]} ;
end
m2_st41 :
begin
m2_nexts <= m2_st42 ;
q_data4_x <= {ps2_data, q_data4_x[8:1]} ;
end
m2_st42 :
begin
m2_nexts <= m2_st43 ;
q_data4_x <= {ps2_data, q_data4_x[8:1]} ;
end
m2_st43 :
begin
m2_nexts <= m2_st44 ;
q_data4_x <= {ps2_data, q_data4_x[8:1]} ;
end
m2_st44 :
begin
m2_nexts <= m2_st45 ;
q_data4_x <= {ps2_data, q_data4_x[8:1]} ; //data7
end
m2_st45 :
begin

```

```

m2_nexts <= m2_st46 ;
q_data4_x <= {ps2_data, q_data1_x[8:1]} ;
q_data4 <= q_data4_x[8:1] ;
end
m2_st46 :
begin
m2_nexts <= m2_st14 ; //stop4
end
default :
begin
end
endcase
end
end
always @(reset or q_data2)
begin : u6
if (reset == 1'b1)
begin
ps2_data1 <= {8{1'b0}} ;
end
else
begin
ps2_data1 <= q_data2 ;
end
end
endmodule

```

5、管脚约束:

```

set_pin_assignment { CLK } { LOCATION = R7; IOSTANDARD = LVCMOS33; PULLTYPE = PULLUP; }
set_pin_assignment { PS2CLK } { LOCATION = M16; IOSTANDARD = LVCMOS33; PULLTYPE =
PULLUP; }
set_pin_assignment { PS2DATA } { LOCATION = K16; IOSTANDARD = LVCMOS33; PULLTYPE =
PULLUP; }
set_pin_assignment { RST } { LOCATION = A2; IOSTANDARD = LVCMOS33; PULLTYPE = PULLUP; }
set_pin_assignment { SPK } { LOCATION = H11; IOSTANDARD = LVCMOS33; DRIVESTRENGTH = 8;
PULLTYPE = NONE; }
set_pin_assignment { code[0] } { LOCATION = A4; IOSTANDARD = LVCMOS33; DRIVESTRENGTH =
8; PULLTYPE = NONE; }
set_pin_assignment { code[1] } { LOCATION = A3; IOSTANDARD = LVCMOS33; DRIVESTRENGTH =
8; PULLTYPE = NONE; }

```

```
set_pin_assignment { code[2] } { LOCATION = C10; IOSTANDARD = LVCMOS33; DRIVESTRENGTH = 8; PULLTYPE = NONE; }
```

```
set_pin_assignment { code[3] } { LOCATION = B12; IOSTANDARD = LVCMOS33; DRIVESTRENGTH = 8; PULLTYPE = NONE; }
```

6、下载测试：

将 PS/2 键盘连接到扩展模块对应 PS/2 接口。进行程序下载，程序下载完毕后，将 SW5 开关拨上（BUZ）端。当按下键盘上方一排 0-9 按键时，蜂鸣器演奏对应音符，此时 LED 灯会显示当前按下的音符对应的值。

